Policy Gradient Methods

Alvaro Soto

Computer Science Department (DCC), PUC

DQN requires a large replay buffer

- Fills buffer *offline* before learning $\Rightarrow$ costly in slow/expensive environments
- Training/deployment bottleneck: robotics, clinical trials, etc.

Q-function is only a proxy

- We want π^* but we learn Q^* first, two main inconveniences:
  - Errors in Q propagate to π via $\arg\max$
  - In the discrete case, $\arg\max_a Q(s, a)$ is not a main complication, however, in continuous action spaces (ex. robotics apps), this requires a costly optimization.

Learning a policy directly: Planning to act in the world

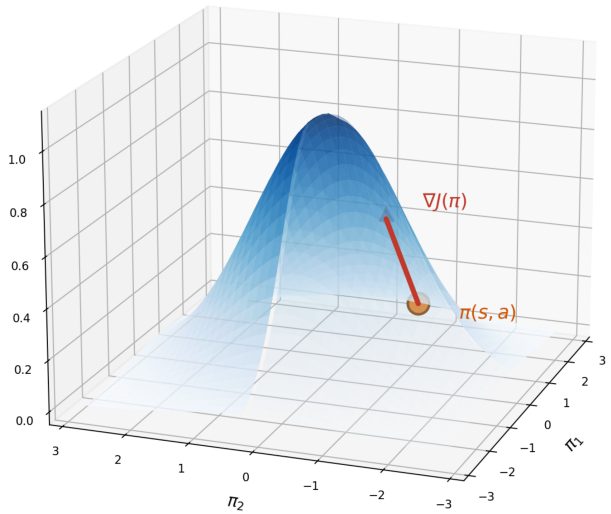
Objective Function:

Maximize expected cumulative and discounted reward

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t \, r_t\right]$$

This is equivalent to optimize the value function averaged over the start-state distribution: $J(\theta) = \mathbb{E}_{s_0}[V^{\pi_\theta}(s_0)]$

We will optimize the underlying policy using gradient ascent



Key question: how do we compute $\nabla_\theta J(\theta)$?

(ex. there is not a transition function to compute the derivative!)

Policy: $\pi_\theta(a|s)$ (e.g. a neural network with weights θ)

Data given by episodes or trajectories: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$

Reward of a trajectory:

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t \, r_t$$

Observation: sum goes to $T-1$ because in our notation S_T is the terminal state.

Objective Function

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] = \sum_{\tau} P(\tau;\theta)\, G(\tau)$$

$P(\tau;\theta)$: probability of trajectory τ under policy π_θ

Goal: find $\theta^* = \arg\max_\theta J(\theta)$ using gradient ascent

Gradient ascent: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Challenge: compute gradient of the objective function:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta)\, G(\tau) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

As usual in RL, we assume a MDP structure for $P(\tau;\theta)$, where:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$$

Then:

$$P(\tau;\theta) = p(s_0)\pi_\theta(a_0|s_0)p(s_1|a_0,s_0) \;\; \ldots \;\; \pi_\theta(a_{T-1}|s_{T-1})p(s_T|a_{T-1},s_{T-1})$$

Observation. Notice a key subtlety, we extend classic RL, where the policy is a deterministic function, to a more general case, where the policy is a probability distribution (PDF).

Policy as a Probability Distribution

Classic RL view: the policy is a *deterministic* mapping from states to actions:

$$a_t = \pi(s_t)$$

Policy gradient view: it uses a more general definition where the policy is a *probability distribution* over actions, conditioned on the state:

$$\pi_\theta(a \mid s) : \mathcal{S} \to \Delta(\mathcal{A})$$

Key subtlety: it is a Markov kernel, not one distribution

$\pi_\theta(a \mid s)$ is **not** a single probability distribution. For each fixed state s, $\pi_\theta(\cdot \mid s)$ is a distribution over the action space. The policy is the whole *family* $\{\pi_\theta(\cdot \mid s)\}_{s \in \mathcal{S}}$.

Why this matters:

- Stochasticity gives natural *exploration* (no ϵ-greedy needed).
- As we will see, $\pi_\theta(a \mid s)$ is differentiable in θ.
- Works for *continuous* action spaces, where $\arg\max_a Q(s, a)$ was the costly bottleneck (slide 2).

Going back to our challenge: $\nabla_\theta J(\theta)$

We need to compute:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta)\, G(\tau) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

where,
$P(\tau;\theta) = p(s_0)\pi_\theta(a_0|s_0)p(s_1|a_0,s_0) \;\; \ldots \;\; \pi_\theta(a_{T-1}|s_{T-1})p(s_T|a_{T-1},s_{T-1})$

Problem: we cannot calculate $\nabla_\theta P(\tau;\theta)$ directly. Transition function $p(s_t|a_{t-1},s_{t-1})$ is unknown

Key insight: Log-derivative trick

$$\nabla_\theta \log f(x) = \frac{\nabla_\theta f(x)}{f(x)} \quad \Longrightarrow \quad \nabla_\theta f(x) = f(x)\, \nabla_\theta \log f(x)$$

Apply with $f(x) = P(\tau;\theta)$:

$$\nabla_\theta P(\tau;\theta) = P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta)$$

We need:

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

and

$$\nabla_\theta P(\tau;\theta) = P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta)$$

then

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\, P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)\, \nabla_\theta \log P(\tau;\theta)]$$

$$P(\tau;\theta) = p(s_0)\pi_\theta(a_0|s_0)p(s_1|a_0,s_0) \;\; \ldots \;\; \pi_\theta(a_{T-1}|s_{T-1})p(s_T|a_{T-1},s_{T-1})$$

$$\log P(\tau;\theta) = \log p(s_0) + \sum_{t=0}^{T-1} \underbrace{\log \pi_\theta(a_t|s_t)}_{\text{depends on } \theta} + \sum_{t=0}^{T-1} \underbrace{\log p(s_{t+1}|s_t,a_t)}_{\text{does not depend on}\theta}$$

The environment terms (transition functions) vanish when we take ∇_θ:

$$\nabla_\theta \log P(\tau;\theta) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Almost there ... putting all together

Our original objective is : $J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t \, r_t\right]$

Using as a proxy the expected return over trajectories τ sampled from π_θ:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] = \sum_\tau P(\tau;\theta)\, G(\tau)$$

Optimizing using gradient ascent, and applying the gradient trick:

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\nabla_\theta P(\tau;\theta) \; = \sum_\tau G(\tau)P(\tau;\theta)\nabla_\theta \log P(\tau;\theta)$$

Applying the vanishing environment result:

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)P(\tau;\theta) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[G(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\ \underbrace{G(\tau)}_{\substack{\text{scalar:}\\ \text{how good was } \tau}}\ \cdot\ \underbrace{\textstyle\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t|s_t)}_{\substack{=\nabla_\theta \log P(\tau;\theta)\\ \text{direction that makes } \tau \text{ more likely}}}\ \Big]$$

Main result and the basis of all subsequent RL policy gradient methods.

- The gradient is an expectation
- Expectation is over episodes τ sampled from π_θ

The expectation has two main terms:

- $\nabla_\theta \log P(\tau;\theta)$: the direction in parameter space that **increases the probability of the whole trajectory** τ
- $G(\tau)$: a scalar that **scales that push**. High return $\Rightarrow$ strong push to make τ more likely; low or negative return $\Rightarrow$ weak push, or a push away

In summary: probability mass moves toward high-return trajectories. Trial and error, made differentiable.

- No gradient through the environment or the reward, $G(\tau)$ is just a number
- We only need π_θ to be differentiable. Rewards can be non-differentiable. The environment (or simulator) can be a black box.

Some relevant observations

1. **The samples come from π_θ itself (on-policy).** Every update changes the distribution we sample from, so old episodes become obsolete
2. **Credit assignment is crude.** Every action in τ receives the same $G(\tau)$, including rewards obtained *before* that action was taken.

Estimating the Gradient using Monte Carlo

The gradient is an expectation, we can use sampling to obtain an estimation $\nabla_\theta \hat{J}(\theta)$ using N episodes $\tau^{(1)}, \ldots, \tau^{(N)}$:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[G(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left[G(\tau^{(i)}) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) \right]$$

1. **Collect** N complete episodes $\tau^{(1)}, \ldots, \tau^{(N)}$ under current π_θ
2. **Compute** return $G(\tau^{(i)})$ for each episode
3. **Compute** weighted average of gradient estimates:
   $\nabla_\theta \hat{J}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left[G(\tau^{(i)}) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) \right]$
4. **Update:** $\theta \leftarrow \theta + \alpha \nabla_\theta \hat{J}(\theta)$

REINFORCE Algorithm (Williams, 1992)

We are now ready to present our first working algorithm: REINFORCE. It implements the previous ideas with two refinements:

- Collecting N trajectories for each gradient update can be cumbersome, so it approximates the gradient using a single sampled trajectory per update (N=1).
- It applies a per-step update using the **reward-to-go** $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$ instead of a single aggregated update for the full-episode (improves the credit assignment problem).

Algorithm 1 REINFORCE (Williams, 1992)

Initialize policy parameters θ randomly
repeat
  Generate episode $\tau = (s_0, a_0, r_0, \ldots, s_T)$ following π_θ
  for $t = 0, 1, \ldots, T-1$ **do**
    Compute return: $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$
    Update: $\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)$
  end for
until converged

Calculation of G_t considers only rewards after t. This is a causality issue, action a_t is only responsible of rewards after t.

Key question: how do we compute $\nabla_\theta \log \pi_\theta(a_t|s_t)$?

It is the **gradient of a policy**. In our case, the **gradient of a NN** ?

Connection to Supervised Learning

Recall: in supervised learning with cross-entropy loss $\mathcal{L}_{\text{SL}}(\theta)$:

$$\mathcal{L}_{\text{SL}}(\theta) = -\sum_i \log \pi_\theta(y_i|x_i) \quad \Rightarrow \quad \nabla_\theta \mathcal{L}_{\text{SL}} = -\sum_i \nabla_\theta \log \pi_\theta(y_i|x_i)$$

REINFORCE gradient (per step):

$$\nabla_\theta J(\theta) \ \propto \ \sum_t G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Key mapping

Supervised learning		**REINFORCE**
input x_i	$\longleftrightarrow$	state s_t
label y_i	$\longleftrightarrow$	sampled action a_t
sample weight $= 1$	$\longleftrightarrow$	sample weight $= G_t$

Key Result

Minimizing $\mathcal{L}_{\text{SL}}$ with label a_t and sample weight G_t **is** equivalent to the REINFORCE update

Define the **surrogate loss**:

$$\mathcal{L}_{\text{PG}}(\theta) = -\frac{1}{T}\sum_{t=0}^{T-1} G_t \cdot \log \pi_\theta(a_t|s_t)$$

Why "surrogate"?
$\mathcal{L}_{\text{PG}}$ has the **same gradient** as $J(\theta)$ w.r.t. θ, but is differentiable.

Implementation

- Collect episode τ
- Compute returns $\{G_t\}$
- Treat (s_t, a_t) as labeled data
- Minimize $\mathcal{L}_{\text{PG}}$ with Adam/SGD

In PyTorch

```
log_probs =
policy(states).log_prob(actions)
loss = -(returns *
log_probs).mean()
loss.backward()
```

This is really great! we can implement policy gradient optimization as SL classification, we just need to reweight the cross-entropy by G_t.

REINFORCE: Limitations

1. **On-policy**
   - Must collect new data after *every* update
   - Old episodes cannot be reused: sample-inefficiency
   - We will use Importance Sampling to alleviate this

2. **High variance**
   - Gradient estimation depends only on reward of observed actions
   - High variance due to lack of a baseline to evaluate the real impact of observed vs unobserved actions.
   - We will introduce a baseline to alleviate this

1 An off-policy approach

Core question: Can we reuse data from *old* policies and still optimize the *current* policy safely?

If we use an auxiliary $\pi_{\theta_{\text{aux}}}$ to generate the data, like the old policy $\pi_{\theta_{\text{old}}}$, we need to compensate this fact.

Importance sampling corrects for the distribution mismatch:

$$\mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\frac{P(\tau;\theta)}{P(\tau;\theta_{\text{old}})}G(\tau)\right] = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] = J(\theta)$$

So, we need to introduce a correcting weight (**importance weight**) for each sample. Fortunately, these weights do not depend on the transition function. As in slide 8, the environment cancels again:

$$\frac{P(\tau;\theta)}{P(\tau;\theta_{\text{old}})} = \frac{\cancel{p(s_0)}\prod_t \pi_\theta(a_t|s_t)\cancel{p(s_{t+1}|s_t,a_t)}}{\cancel{p(s_0)}\prod_t \pi_{\theta_{\text{old}}}(a_t|s_t)\cancel{p(s_{t+1}|s_t,a_t)}} = \prod_{t=0}^{T-1}\underbrace{\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}}_{r_t(\theta)}$$

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\prod_{t=0}^{T-1}\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}G(\tau)\right] = \mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\prod_{t=0}^{T-1} r_t(\theta)G(\tau)\right]$$

Problem: This estimator is not practical. A product of T ratios leads to exponential variance growth in T.

- Ex. $T=100$, a uniform $r_t=1.02$ gives weight 7.2, while $r_t=0.98$ gives 0.13.
- A few trajectories take all the weight.

Idea: use one ratio per step instead of a product. Reuse a batch collected with $\pi_{\theta_{\text{old}}}$, but reweight *one transition at a time* instead of whole trajectories,

$$\nabla_\theta J(\theta) \;\approx\; \mathbb{E}_{s_t\sim\rho_{\theta_{\text{old}}},\, a_t\sim\pi_{\theta_{\text{old}}}(\cdot|s_t)}\left[\, r_t(\theta)\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)\,\right], \qquad r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

A *single* ratio per step: no product, so no exponential variance in T.

Obs: states in the batch were visited by $\pi_{\theta_{\text{old}}}$. This holds only while θ stays **close** to θ_{old}. Later, we will introduce a clipping of the policy that will enforce this assumption.

2 Variance reduction

$$\nabla_\theta \log P(\tau;\theta) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Estimation of $\nabla_\theta \log P(\tau;\theta)$ only considers gradient directions at combinations (s_t, a_t) present in the training batch. This can be misleading.

Consider this example: one state s, 3 actions, softmax policy uniform.

$$\pi_\theta = (\tfrac{1}{3}, \tfrac{1}{3}, \tfrac{1}{3}), \qquad \nabla_\theta \log \pi_\theta(a|s) = e_a - \pi_\theta, \qquad G = (100, 101, 102)$$

If the batch contained the 3 actions:

$$100 \begin{pmatrix} \frac{2}{3} \\ -\frac{1}{3} \\ -\frac{1}{3} \end{pmatrix} + 101 \begin{pmatrix} -\frac{1}{3} \\ \frac{2}{3} \\ -\frac{1}{3} \end{pmatrix} + 102 \begin{pmatrix} -\frac{1}{3} \\ -\frac{1}{3} \\ \frac{2}{3} \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \\ +1 \end{pmatrix}$$

Good ranking: lower the logit of a_1, leave a_2, raise a_3

But if the batch only contains a_1:

$$100\,(e_1 - \pi_\theta) = 100 \cdot (\tfrac{2}{3}, -\tfrac{1}{3}, -\tfrac{1}{3}) = (+66.7,\ -33.3,\ -33.3)$$

A large step **increasing** the *worst* action.

Where $\nabla_\theta \log \pi_\theta(a|s) = e_a - \pi_\theta$ comes from

Softmax policy over $|\mathcal{A}|$ actions with logits $z \in \mathbb{R}^{|\mathcal{A}|}$:

$$\pi_\theta(a|s) = \frac{e^{z_a}}{Z}, \qquad \text{with normalizer} \quad Z = \sum_j e^{z_j}$$

Taking $\log$ on both sides, the quotient becomes a difference:

$$\log \pi_\theta(a|s) = z_a - \log Z = z_a - \log \sum_j e^{z_j}$$

Differentiate with respect to one logit z_c. The logits are independent coordinates, so each term produces a Kronecker delta:

$$\underbrace{\frac{\partial z_a}{\partial z_c} = \delta_{ac}}_{\text{selects one component}} \qquad \underbrace{\frac{\partial}{\partial z_c} \log \sum_j e^{z_j} = \frac{\sum_j e^{z_j} \delta_{jc}}{Z} = \frac{e^{z_c}}{Z} = \pi_\theta(c|s)}_{\text{collapses the sum}}$$

$$\frac{\partial \log \pi_\theta(a|s)}{\partial z_c} = \delta_{ac} - \pi_\theta(c|s) \qquad \Longrightarrow \qquad \nabla_z \log \pi_\theta(a|s) = e_a - \pi_\theta$$

Idea: for a given state s, we need to reinforce actions that in average provides reward higher than other actions in that state.

Solution:

- Subtract a reference level $b(s_t)$ that depends on the state only. This defines the **advantage**: $A_t = G_t - b(s_t)$.
- Usually $b(s_t) = V(s_t)$, the value function. Then $A_t = G_t - V(s_t)$ indicates how much better was a_t than what we expect in s_t

Ex.: $\pi_\theta = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$, $G = (100, 101, 102)$,

$V(s) = \mathbb{E}_{a\sim\pi_\theta}[G] = 101$ and $A = (-1,\ 0,\ +1)$.

batch	weight G	weight $A = G - V$
only a_1 (worst)	$(+66.7,\ -33.3,\ -33.3)$	$(-0.67,\ +0.33,\ +0.33)$
only a_2	$(-33.7,\ +67.3,\ -33.7)$	$(0,\ 0,\ 0)$
only a_3 (best)	$(-34.0,\ -34.0,\ +68.0)$	$(-0.33,\ -0.33,\ +0.67)$

The estimator is then:

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{s_t\sim\rho_{\theta_{\text{old}}},\, a_t\sim\pi_{\theta_{\text{old}}}(\cdot|s_t)}\left[\, r_t(\theta)\, A_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)\,\right], \qquad A_t = G_t - V(s_t)$$

Why subtracting a baseline is allowed

The extra term has zero mean. For a fixed state s:

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\left[b(s)\,\nabla_\theta \log \pi_\theta(a|s)\right] = b(s)\sum_a \pi_\theta(a|s)\,\frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} = b(s)\,\nabla_\theta \underbrace{\sum_a \pi_\theta(a|s)}_{=1} = 0$$

- b must **not depend on the action** a: otherwise it cannot leave the sum.
- b is treated as a constant w.r.t. θ: no gradient flows through it (V_ϕ is a separate network).

PPO: Proximal Policy Optimization(Schulman et al., 2017)

PPO uses $b_t = V(s_t)$

> *"Was action a_t better or worse than what we normally expect in s_t?"*

Recall

$$V_\theta^\pi(s_t) = \mathbb{E}_{\pi_\theta}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \;\middle|\; s_t\right]$$

We needs a second NW to estimate $V_\phi(s_t)$

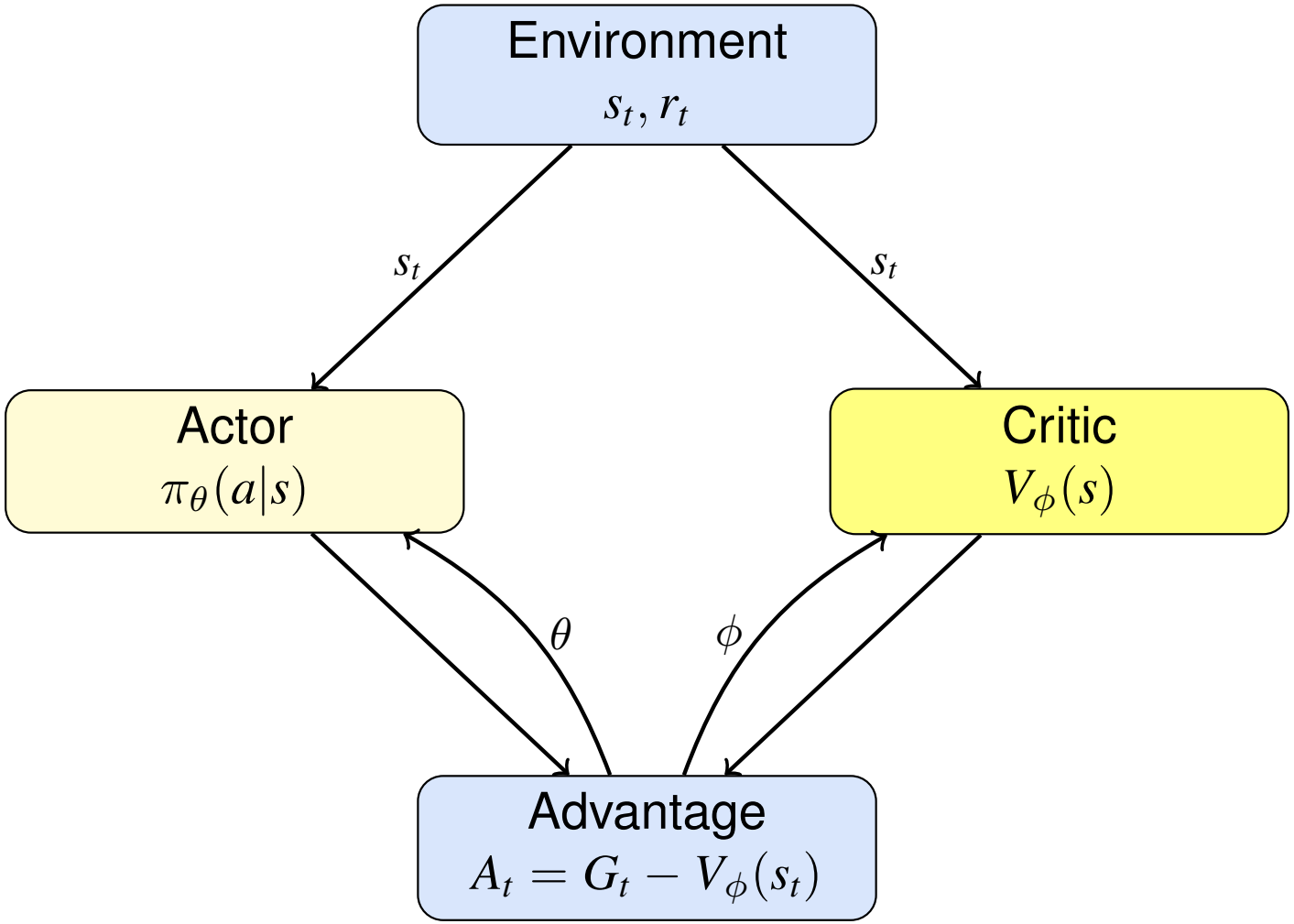
Actor-Critic approach:

- **Critic** learns which states are inherently good or bad
- **Actor** needs to favor actions that are better than expected

The **critic** predict the expected return from each state:

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_t\left[\left(V_\phi(s_t) - G_t\right)^2\right]$$

The **actor** π_θ uses V_ϕ to compute advantages and update the policy.

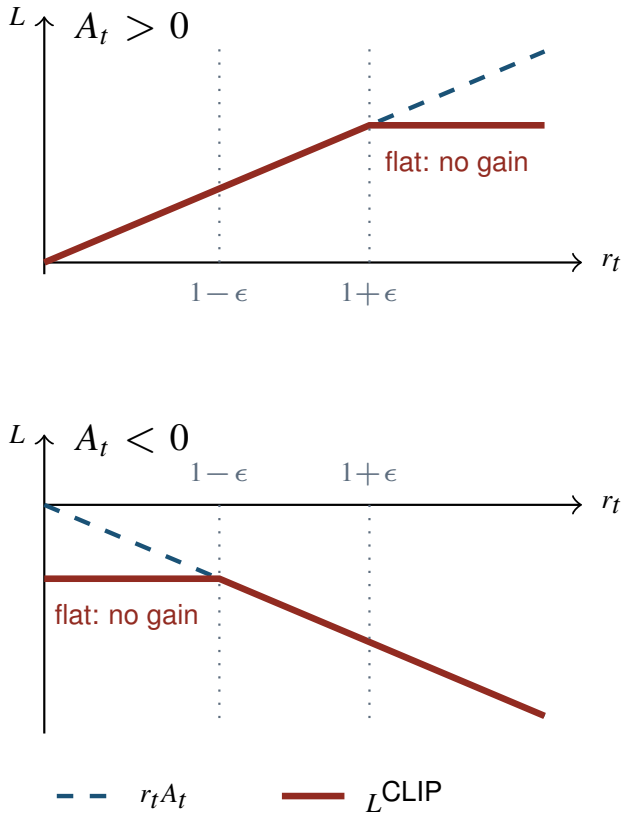

Environment
s_t, r_t
s_t
s_t
Actor
$\pi_\theta(a|s)$
Critic
$V_\phi(s)$
θ
ϕ
Advantage
$A_t = G_t - V_\phi(s_t)$

The Problem with Unconstrained Updates

When using importance sampling, states come from $\pi_{\theta_{\text{old}}}$

If θ moves *too far* from θ_{old}:

- The ratio $r_t(\theta)$ becomes very large or very small
- Importance sampling estimate is no longer reliable
- Gradient steps become destructively large



Key insight (PPO)

We want to take the **largest possible improvement step** without going so far that the policy becomes unreliable.

PPO: Proximal Policy Optimization

Core idea: clip the probability ratio so updates stay close to $\pi_{\theta_{\text{old}}}$.

PPO-Clip Objective (Schulman et al., 2017)

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min\{r_t(\theta)\,A_t,\ \text{clip}(r_t(\theta),\ 1-\epsilon,\ 1+\epsilon)\,A_t\}]$$

where $r_t(\theta) = \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\epsilon \approx 0.2$.

How the clip works:

- If $A_t > 0$ (good action): ratio is clipped at $1+\epsilon$ can only increase probability so much
- If $A_t < 0$ (bad action): ratio is clipped at $1-\epsilon$ can only decrease probability so much
- Takes the **pessimistic** (min) bound conservative by design

PPO Algorithm

Algorithm 2 PPO-Clip (Schulman et al., 2017)

Initialize policy π_θ and value network V_ϕ
repeat
    Collect set of trajectories $\mathcal{D}$ under current π_θ
    Compute advantages $\hat{A}_t$
    for K epochs **do**
        Sample mini-batches from $\mathcal{D}$
        Compute $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$
        Update θ by maximizing $L^{\text{CLIP}}(\theta)$
        Update ϕ by minimizing value loss $(V_\phi(s_t) - G_t)^2$
    end for
    $\theta_{\text{old}} \leftarrow \theta$
until converged

Key difference from REINFORCE: data $\mathcal{D}$ is reused for K gradient epochs before being discarded

REINFORCE vs PPO

Property	REINFORCE	PPO
Data reuse	None (on-policy)	K epochs per batch
Variance	High	Lower
Update stability	Unconstrained	Clipped ratio
Value network	No	Yes (V_ϕ)
Sample efficiency	Low	Moderate
Implementation	Simple	Moderate

Why PPO dominates in practice

PPO keeps the **simplicity of policy gradient** (no Q-function, no replay buffer) while fixing the two main failure modes of REINFORCE: high variance and unstable updates.

PPO in the LLM setting: do we need a critic?

PPO trains **two** networks: the actor π_θ and the critic V_ϕ.

In LLM post-training (math, code, reasoning) this is expensive:

- V_ϕ is usually a network of the *same scale* as the policy $\Rightarrow$ roughly doubles memory and compute.
- The reward arrives only at the end of the sequence: a verifier (or a reward model) scores the complete answer.
- So V_ϕ must regress a terminal-only signal at every intermediate token: a hard and noisy learning problem.

The critic was never the goal

V_ϕ is just *one* way of producing a baseline $b(s)$. The only conditions we actually needed were: b must not depend on a, and b must not depend on θ. Nothing forces b to be a learned function.

Key question: can we estimate $b(s)$ *directly from samples*?

An LLM is already an MDP

An LLM can be considered a stochastic policy to generate text:

- Given **Prompt** q, model generates a completion $o = (o_1, \ldots, o_{|o|})$, one token at a time.
- **State:** $s_t = (q, o_{<t})$
- **Action:** $a_t = o_t \in \mathcal{V}$ ($\mathcal{V}$: vocabulary)
- **Policy:** $\pi_\theta(o_t \mid q, o_{<t})$, it predicts the next token
- **Transition:** deterministic, no environment dynamics, just append the next token (dialog/tools are exceptions)
- **Reward:** $r(q, o)$, usually given once, when the answer is complete. So $\gamma = 1$ is natural, every token's return is r, regardless of length

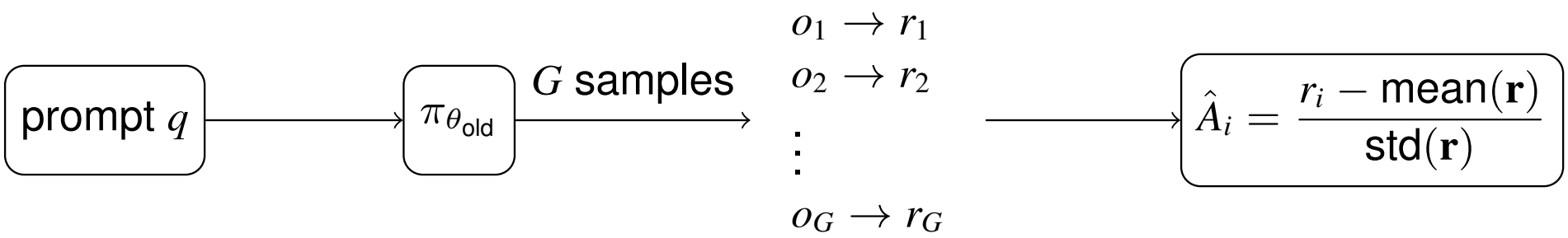
Relevant insights

- Discrete actions: acting is one forward pass plus sampling from a softmax over $|\mathcal{V}| \approx 10^5$ tokens
- No per-step reward, a per-token critic must learn from a terminal-only signal
- What remains from PPO is: the ratio $r_t(\theta)$, the clip, and the *baseline*.
- Hard part moves to *exploration*: $|\mathcal{V}|^L$ possible answers L: max length), explored only through sampling

GRPO objective (Shao et al., 2024)

IDEA: *"was this answer better or worse than the other answers the model itself produces for this same prompt?"*

For a given prompt q, sample a **group** of G completions from the old policy and score all of them:



The baseline is the group mean: $\boldsymbol{b(q) \approx \frac{1}{G} \sum_{j=1}^{G} r_j}$

- $b(q)$ is a Monte Carlo estimate of $V(q)$, computed from samples instead of from a network
- Every token of o_i inherits the same advantage: $\hat{A}_{i,t} = \hat{A}_i$ for all t.

GRPO objective (Shao et al., 2024)

GRPO

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q,\ \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \Big[\ell_{i,t}(\theta) \ - \ \beta\, \mathbb{D}_{\text{KL}}[\pi_\theta \,\|\, \pi_{\text{ref}}] \Big]$$

$$\ell_{i,t}(\theta) = \min \Big\{ r_{i,t}(\theta)\, \hat{A}_{i,t},\ \text{clip}\big(r_{i,t}(\theta),\ 1-\epsilon,\ 1+\epsilon\big)\, \hat{A}_{i,t} \Big\}, r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}$$

What is inherited from PPO: the per-token ratio $r_{i,t}$, the clip mechanism, and K gradient epochs over the same batch.

What changes: $\hat{A}_{i,t}$ no longer comes from V_ϕ, it comes from the group; and the KL to a *reference* model is added explicitly.

With $\beta = 0$ and $\hat{A}_{i,t}$ from a critic, we are back to PPO.

The KL term: staying close to the reference model

$$\mathbb{D}_{\mathrm{KL}}[\pi_\theta \,\|\, \pi_{\mathrm{ref}}] = \frac{\pi_{\mathrm{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - \log \frac{\pi_{\mathrm{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q, o_{i,<t})} - 1 \;\geq\; 0$$

- Estimated from the *sampled* token only: unbiased and, unlike $-\log(\pi_{\mathrm{ref}}/\pi_\theta)$, guaranteed non-negative (Schulman's k_3 estimator) $\Rightarrow$ much lower variance.
- π_{ref} is the frozen SFT model. β controls how far the policy is allowed to drift from it.
- **Two different constraints, do not confuse them:**
  - the clip keeps θ close to θ_{old} (validity of the importance-sampling estimator);
  - the KL keeps θ close to θ_{ref} (preserve fluency, avoid reward hacking).
- Note: in classic RLHF-PPO the KL penalty is usually folded into the *reward*; GRPO puts it directly in the loss.

GRPO Algorithm

Algorithm 3 GRPO (Shao et al., 2024)

Initialize policy π_θ from the SFT model; set $\pi_{\text{ref}} \leftarrow \pi_\theta$
repeat
    $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
    Sample a batch of prompts $\{q\}$
    for each prompt q **do**
        Sample G completions $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q)$
        Score them: $r_i = r(q, o_i)$ ▷ verifier or reward model
        $\hat{A}_{i,t} \leftarrow \big(r_i - \text{mean}(\mathbf{r})\big)/\text{std}(\mathbf{r})$ for all t
    end for
    for K epochs **do**
        Update θ by maximizing $\mathcal{J}_{\text{GRPO}}(\theta)$ on mini-batches
    end for
until converged

Fine print

- **The baseline is slightly biased.** mean$(\mathbf{r})$ includes r_i itself, so b is not independent of the sampled action and the zero-mean argument does not hold exactly. A leave-one-out mean (RLOO) restores unbiasedness; in practice with moderate G the bias is $O(1/G)$ and is accepted
- **Dividing by std$(\mathbf{r})$ is a per-question rescaling.** It inflates the gradient of questions where all G answers are almost equally good (or bad), i.e. exactly the least informative ones. *Dr. GRPO* (Liu et al., 2025) removes it
- **The $1/|o_i|$ factor introduces a length bias:** it makes each token of a long answer count less, which asymmetrically discounts long correct answers and long wrong ones. *DAPO* (Yu et al., 2025) normalizes by the total number of tokens in the batch instead
- **Degenerate groups:** if all r_i are equal, then $\hat{A}_i = 0$ for the whole group and the prompt contributes *nothing* to the gradient. DAPO's dynamic sampling resamples such prompts
- **Cost moves from training to inference:** G completions per prompt (typically $G = 8$ to 64), but it parallelizes well and needs no gradients.

PPO vs GRPO

Property	**PPO**	**GRPO**
Baseline	learned critic V_ϕ	group mean over G samples
Networks in memory	2 (+ ref)	1 (+ ref, no gradients)
Credit assignment	per token (via V_ϕ)	per sequence
Reward granularity	any	terminal / sequence-level
Extra sampling	—	G completions per prompt
Update stability	clipped ratio	clipped ratio + KL to π_{ref}
Typical use	control, robotics	LLM post-training

Why GRPO took over LLM post-training

When the reward is a single verifiable scalar per answer, a per-token critic buys little and costs a lot. Replacing it with a Monte Carlo estimate over several answers to the *same* prompt keeps the whole policy-gradient machinery intact and halves the training footprint. It is the algorithm behind DeepSeekMath and DeepSeek-R1.

Where the field is moving (optional slide)

- **RLOO** — leave-one-out baseline; unbiased version of the same idea.
- **Dr. GRPO** — removes $/\text{std}$ and the length normalization to eliminate the two biases above.
- **DAPO** — decoupled ("clip-higher") bounds, dynamic sampling, token-level loss, no KL.
- **GSPO** — importance ratio defined at the *sequence* level instead of per token, which is better matched to a sequence-level reward.

All of them are the **same policy gradient theorem** of slide 9, with different answers to two questions:

what do we use as a baseline? and *how do we keep the update small?*